

Agent libOS: A Runtime Substrate for Capability-Controlled Self-Evolving LLM Agents

Yingqi Zhang

Department of Computer Science and Technology, Tsinghua University
Beijing, China
zhangyq24@mails.tsinghua.edu.cn

Abstract

Large language model (LLM) agents increasingly persist across tasks, acquire memory, activate Skills, synthesize tools, fork child processes, attach remote resources, and commit checkpoints as reusable images. These mechanisms expand what an agent can request after deployment. If a runtime treats an action’s visibility as authorization to perform it, however, self-evolution becomes a path to authority escalation and data exfiltration.

This paper presents Agent libOS, an agent-native library OS substrate that separates three planes. The design defines *operation admission* as the conjunction of process identity, the applicable Task Authority ceiling, typed Capabilities, deterministic policy or Human approval, resource budgets, and execution through a concrete primitive. *Information-flow admission* propagates Runtime-maintained labels and immutable source references, resolves a canonical Sink from the Host-owned registry, checks its clearance, and, for conditionally permitted high-sensitivity egress, requires an exact one-shot Human release. A separate *evidence plane* records durable intent, classified outcomes, events, audit records, and causal links; evidence explains a decision but never grants authority. The resulting invariant is that an agent’s model-visible action surface may evolve without implicitly expanding either its resource authority or its permitted information flows.

The implementation includes persistent AgentProcess instances with process-local Object Memory namespaces and working directories; declarative Skills; syscall-mediated Deno/TypeScript JIT Tools; images and scoped checkpoints; typed Git; client-only JSON-RPC and MCP providers; Human queues; hierarchical resource accounting; Host-only Explainable Operations; and Durable Task Run supervision with fail-closed recovery. Provider-backed effects follow a prepare–dispatch–settle protocol that makes ambiguous dispatched outcomes explicit and does not blindly replay them during recovery. In the source-bound artifacts, the full-Runtime evaluation reports 33/33 task-success oracles and 33/33 safety-passing tasks, with unauthorized-effect and false-denial fractions of 0/119 and 0/119, respectively. The real-model evaluation comprises 42 trajectories. Across the canonical workflow matrix, observed safety is 12/12 and observed strict utility is 12/12. In the paired projection study, the observable-state oracle passes for 15/15 runs in the Skills arm and for 15/15 runs in the baseline; fully correct runs number 13/15 and 13/15, respectively. These results are descriptive of the evaluated model/provider configuration, not population-level rates. The system does not claim to eliminate prompt injection, provide kernel-grade sandboxing, roll back irreversible external effects, or make local evidence cryptographically tamper-proof. Its

contribution is a runtime substrate for agents whose affordances and state can change after deployment.

CCS Concepts

- **Security and privacy** → **Authorization; Information flow control**; • **Computer systems organization** → *Operating systems*;
- **Computing methodologies** → *Artificial intelligence*.

Keywords

LLM agents, capability systems, information flow control, library operating systems, self-evolving agents, durable execution, runtime security

1 Introduction

LLM agents are moving from short chat loops toward long-running software actors. A coding or research agent may maintain task memory across sessions, activate a reusable Skill, register a self-authored Deno/TypeScript JIT Tool, fork a child process, execute a shell command, request Human approval, checkpoint its state, resume after a failure, and later spawn or exec from a checkpoint-derived image. Research on self-evolving agents makes this trend explicit: systems now adapt prompts, memory, tools, workflows, and agent code both within and across task episodes [12, 35, 48, 60–62].

At the model–tool boundary, many agent systems implement variants of a recurring loop: expose action schemas, obtain a model-selected action, dispatch host functionality, return the result, and continue. ReAct, Toolformer, AutoGen, MetaGPT, SWE-agent, and related systems demonstrate the utility of model-directed tool use and structured agentic execution [17, 40, 54, 56, 57]. When treated as the primary runtime boundary, however, this interface can conflate several questions that must remain distinct in a persistent, self-modifying system:

- (1) Which action descriptions are visible to the model?
- (2) Which operation may this process invoke on which resource?
- (3) Which labeled information may flow to which recipient?
- (4) Which durable records explain what was attempted and what happened?

Tool visibility answers only the first question. A schema can guide a model’s action selection, but it is neither an unforgeable authority token nor a data-flow policy, and it does not prove that an external effect committed.

This distinction matters in the presence of indirect prompt injection. A repository file or retrieved page can instruct an agent to activate a “debugging” Skill, register a generated Tool, send context to a remote endpoint, and preserve the result in a reusable

image. Existing work shows that ordinary-looking retrieved text can redirect tool-using agents toward harmful actions [14, 37, 58]. Prompt defenses and planner-executor separation can reduce this attack surface, but a runtime still needs a lower-level boundary that remains stable as the model’s action vocabulary changes. Confirmation wrappers are insufficient when the check resides in the wrapper rather than at the resource-touching primitive. Host isolation alone likewise neither attributes an effect to the responsible process nor links the corresponding provider outcome to the Capability, Task Authority ceiling, policy or Human approval, and any exact release that admitted the operation.

Agent libOS starts from a narrow systems principle:

A self-evolving agent may change what it can ask for, but it cannot thereby change what it is authorized to affect or where its information may flow.

The design expresses this principle through two admission planes and a separate evidence plane (Figure 1). Operation admission requires the conjunction of process identity, the applicable Task Authority ceiling, Capabilities, deterministic policy or Human approval, resource accounting, and a typed primitive. Information-flow admission propagates Runtime-maintained labels and immutable source references, resolves a canonical Sink through the Host-owned registry, checks its clearance, and, for conditionally permitted high-sensitivity egress, requires an exact one-shot Human release. The evidence plane links prepared intent, dispatch, classified outcomes, resource accounting, events, and audit records, but cannot make a denied action admissible.

This paper makes four contributions.

- (1) **A three-plane runtime model.** We distinguish mutable affordances from resource authority, information-flow permission, and evidence. We state admission and recovery invariants for persistent, self-evolving processes.
- (2) **Conjunctive admission and exact release.** We combine typed Capability checks with applicable Task Authority ceilings, policy or Human approval, hierarchical budgets, trusted data labels, a Host-owned Sink registry, and exact one-shot release. Neither plane can supply a missing predicate in the other.
- (3) **Crash-aware protected effects.** Provider-backed operations durably prepare intent and reserve finite authority before dispatch, classify outcomes as terminal or ambiguous, and, when a provider contract supplies an explicit reconciliation hook, reconcile receipts upon reopening without blind replay. The Explainable Operations layer links this protocol to LLM, Tool, syscall, primitive, Capability, Human, provider-effect, resource, event, and audit evidence.
- (4) **A source-bound evaluation design.** The artifact separates deterministic authority tests, native-live and modeled practical workflows, structural recovery gates, a canonical real-model durable-workflow gate, and a paired comparison of Skill projection with a full-schema baseline into distinct evidence classes. Every quantitative claim is generated from hashed reports bound to one clean source revision; real-model reports are additionally bound to their provider configuration.

The claim is architectural rather than planner-centric. Agent libOS proposes neither a new reasoning prompt nor an optimizer, and

it does not assume that the model reliably recognizes malicious instructions. It aims to mediate resource effects and runtime-delivered information flows through explicit Host-composed boundaries.

2 Threat Model and System Model

2.1 Assets, adversaries, and trusted components

Protected assets include filesystem and Git state; shell and PTY execution; Object Memory and prompt context; Human interactions; process and image lifecycles; checkpoints; provider credentials and configuration; registered remote providers and the systems they reach; LLM calls; finite resource budgets; labeled data; and the durable records required to explain or recover operations. The adversary may control model output, repository files, retrieved documents, Tool results, generated Skill text, JIT source, remote-provider responses, Human free-text answers, process messages, or other content presented to a model. It may know Tool names, Object identifiers, endpoint identifiers, namespace strings, checkpoint identifiers, and resource paths. It may induce the model either to request a dangerous action or to compose several actions that appear individually plausible but are dangerous in combination.

The trusted computing base (TCB) comprises Runtime managers, Host-side composition logic, trusted issuers, the Human operator when an approval or release is accepted, the configured Resource Provider Substrate, accepted Runtime Modules and their source-identity bindings, registered provider policies, the Python interpreter, the storage backend, and the host OS. Each provider is trusted only to the extent required by its declared boundary. For example, an MCP stdio executable runs as native code under the Host account. Skills, JIT code, model output, workspace-supplied image definitions, repository contents, and remote payloads are not trusted to enforce policy. An administrator with direct database access can alter local evidence and therefore remains within the TCB. When the bundled GUI is enabled, its Electron main/preload-renderer stack, packaged assets, and dependency supply chain also enter the TCB because they can reach Host/operator surfaces.

The system is designed to mediate operations and information delivered through the Runtime. It does not claim to prevent untrusted text from changing a model’s plan; prevent a trusted recipient from forwarding data after delivery; encrypt data at rest; provide a kernel, hypervisor, or verified sandbox; compensate for irreversible provider effects; or expose a public remote administration API. These scope boundaries matter: a Capability denial is evidence only that a particular mediated action was denied, not that every possible effect on the host is impossible.

2.2 Runtime state

We model each process as a runtime subject with state

$$p = \langle id, img, status, mem, cwd, T, S, J, C, M, \mathcal{B}, \mathcal{L}, Q, children \rangle. \quad (1)$$

Here, *id* identifies the process; *img* is its AgentImage; and *status* is its lifecycle state. The variables *mem* and *cwd* denote process-local views of Object Memory and the working directory, respectively. The variables *T*, *S*, and *J* denote the callable Tool table, loaded Skills, and registered JIT Tools; *C* is the Capability set; *M* is the active Task Authority Manifest; *B* is the hierarchical budget state;

$\mathcal{L}$ contains the labels and source references associated with the process’s current data; Q is a durable message queue; and *children* records the process-tree relation. Explicit lifecycle states represent spawn, fork, exec, wait, signal, pause, resume, and exit operations instead of encoding them as chat metadata.

An AgentImage defines the default boot-prompt composition, the complete initial Tool table, the narrower initial model projection, default Skills and JIT candidates, LLM-profile selection, context policy, required startup modules, and optional workspace materialization. Required Capabilities declared by an ordinary image serve as diagnostics for Host review, not as grants or launch gates. The narrow image-package bootstrap exception may materialize a fresh private workspace and issue only the filesystem grants specified by the Host-accepted package manifest, and only over that copy. It grants no authority over an existing workspace. Restoring a checkpoint cannot mint authority declared by the image.

The *model-visible action surface* of p is

$$A_p = V(I_p, S_p, J_p, I_p, K_p, R_p), \quad (2)$$

where V computes the current model projection; I_p and K_p denote image and checkpoint affordances; and R_p denotes registered remote affordances. The complete callable table is maintained separately from V : a Skill can project a bounded subset of built-in Tools on demand, while entries hidden from the model remain callable by trusted direct workflows.

Resource authority is distinct from A_p . It comprises the operations admitted by C_p under typed resource matching, effects, constraints, expiration, finite-use limits, and delegation lineage. For an explicit Host manifest and any manifest derived from it, a durable transition_ceiling further intersects that authority during agent-controlled transitions. Let C_p^{ext} denote authority over pre-existing or external resources. Writing Δ^+ for an expansion of a set or a relaxation of a ceiling,

$$\Delta^+ A_p \Rightarrow \Delta^+ C_p^{\text{ext}} \quad \wedge \quad \Delta^+ A_p \Rightarrow \Delta^+ \mathcal{M}_p. \quad (3)$$

Skill activation, JIT registration, image construction, child creation, checkpoint restoration, and endpoint discovery can change A_p , but none of these transitions implicitly expands authority over pre-existing or external resources. When a process creates a new Object, the Runtime may issue the creator a handle to that newly created, owned resource. The trusted image-package bootstrap may similarly issue the bounded private-workspace grants described above. These narrowly scoped fresh-resource grants do not constitute ambient permission administration.

2.3 Three planes

For a normalized operation request r issued by p , the intended operation-admission predicate is

$$\begin{aligned} \text{OpAdmit}(p, r) \equiv & \text{live}(p) \wedge \text{cap}(C_p, r) \\ & \wedge \text{ceiling}(\mathcal{M}_p, r) \wedge \text{policyOrHuman}(p, r) \quad (4) \\ & \wedge \text{budget}(p, r) \wedge \text{primitive}(r). \end{aligned}$$

All predicates must hold. An allow Capability cannot exceed an applicable Task Authority ceiling; Human approval cannot compensate for a missing Capability; a budget does not authorize the operation it measures; and resource identity is normalized and checked at the primitive boundary. The ceiling predicate imposes a

restriction when $\mathcal{M}_p$ is an explicit transition_ceiling manifest or is derived from one. Every launch also receives a durable implicit compatibility record. That record binds launch and model-request evidence but does not constrain authority that the Host deliberately grants later.

For runtime-mediated delivery of payload descriptor x to canonical Sink s , let $\mathcal{V}$ denote the Host-resolved source-version set and let ℓ denote its aggregate labels. Information-flow admission is

$$\begin{aligned} \text{FlowAdmit}(p, r, x, s) \equiv & \text{sources}(x) = \mathcal{V} \wedge \text{labels}(\mathcal{V}) = \ell \\ & \wedge (\text{clear}(s, \ell) \vee \\ & \quad \text{consume}(\text{release}(p, x, s, \mathcal{V}, \ell))) \quad (5) \\ & \wedge \text{integrity}(\ell) \succeq \text{minIntegrity}(r). \end{aligned}$$

The Host, not the model, establishes Sink identity and trust. Labels record sensitivity, tenant and principal identities, trust, and integrity. An exact release is bound to the actor, Sink, payload identity, source versions, label set, Task Authority Manifest, and Sink-registry generation. The release is consumed by the corresponding dispatch and leaves the source labels unchanged. The minimum integrity requirement is independent of sensitivity clearance: neither trusting a Sink nor granting a sensitivity release can endorse low-integrity data.

Internal process handoff is deliberately not treated as delivery to a cleared external Sink. Messages retain their labels and must satisfy the receiving process’s Task Authority receive domain. A child or sibling therefore cannot serve as an implicit declassifier.

The evidence plane comprises causal operations O , events E , audit records D , provider-effect state F , context manifests X , and resource settlement U . None of these records is an admission credential:

$$\forall z \in O \cup E \cup D \cup F \cup X \cup U, \quad \neg \text{credential}(z). \quad (6)$$

Evidence may support or refute a claim, diagnose a failure, or aid reconciliation; its presence may also show that admission occurred earlier. It cannot itself satisfy a missing admission predicate.

2.4 Safety invariants

We structure the implementation around four invariants that can be checked at primitive and recovery boundaries.

I1: no implicit authority over existing resources. Equation 3 holds for every agent-controlled transition that changes affordances. Child delegation attenuates authority. A checkpoint fork intersects restored authority with both current source authority and any applicable explicit or derived Task Authority ceiling; revocations override stale restored state. Authority may nevertheless grow through deliberate later Host grants under an implicit root record, handles for newly created Objects, or the trusted private-workspace bootstrap. In each case, the growth results from an explicit Host or Runtime policy decision, not from growth of the model-visible action surface.

I2: conjunctive dispatch. The design requires every provider dispatch to satisfy Equation 4. If the operation’s contract declares egress, the dispatch must also satisfy Equation 5. The Runtime revalidates all state-dependent bindings at dispatch time rather than relying solely on an earlier preview. Section 7 reports the bound artifact’s coverage and known exception to this requirement.

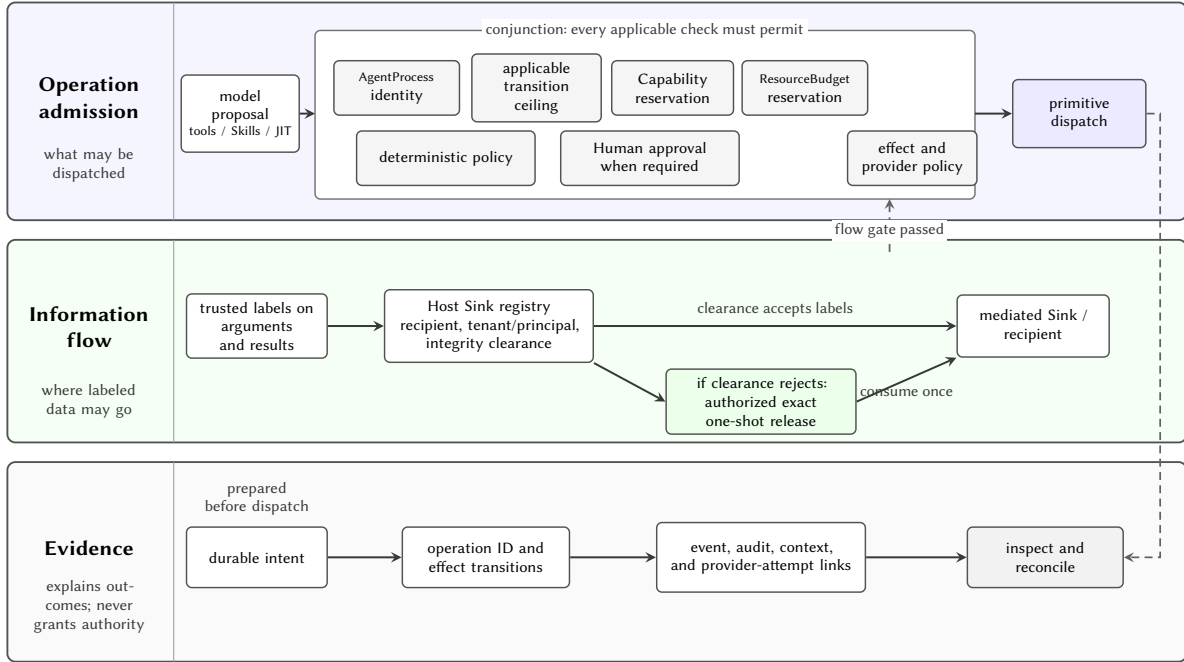


Figure 1: The three-plane runtime boundary. Mutable model-visible affordances determine which operations the model can request; operation admission and information-flow admission remain distinct gates at concrete primitives, each enforced as a conjunction of Host-side checks. Durable evidence records and links decisions without conferring authority.

I3: durable intent and terminal linkage. Each logical protected operation establishes exactly one durable intent identity before its first provider dispatch; all ordered provider phases share that identity. Every terminal outcome finalized by the Runtime includes all contract-required transition, event, audit, causal, authority/release, and resource links. A crash before finalization leaves a pending or unknown recovery input rather than a fabricated terminal record. Evidence completeness is a provenance property: it is neither a safety score nor proof that the provider behaved honestly. We exclude from *provider dispatch* the single checker-pinned Semantic Git preflight exception. Before ordinary operation authority has been established and an intent can be created, this trusted local preflight may perform only a read-only security observation. It returns bounded digests and flow metadata; a subsequently admitted Git operation repeats the observation within the ordinary Protected Operation.

I4: reopen is not redispach. If a crash leaves an effect dispatched but without a certified terminal outcome, its status remains unknown. Upon reopening, the Runtime may repair locally prepared state. It may query a stable provider receipt only if the corresponding contract installs an explicit reconciliation hook. It may neither infer non-execution from the absence of a return value nor blindly replay the provider call. A linked Task Run may independently project the unresolved effect as `needs_attention`.

3 Authority and Information-Flow Admission

3.1 Capabilities and Task Authority

A Capability identifies a typed resource, a set of rights, an effect (allow, ask, or deny), constraints, issuer and holder identities, optional expiration and a finite-use count, and delegation lineage. Resources are typed identifiers rather than free-form wrapper labels. Examples include workspace-relative filesystem paths, pinned repository identities and Git operations, Object Memory namespaces and Objects, process and image subjects, registered JSON-RPC methods and MCP items, shell policies, Human channels, and checkpoints. Filesystem and Git identities are canonicalized before authorization. JSON-RPC and MCP use a two-stage, metadata-hiding gate: public endpoint or server identifiers and public item identifiers first undergo a coarse invocation-authority check. Only an authorized caller may resolve hidden registry metadata; the primitive then performs exact authorization and revalidates the captured specification digest and registry generation immediately before dispatch. Explicit deny overrides allow, and delegation may preserve or narrow, but never widen, a parent grant.

Capabilities determine whether a subject may request an operation on a resource; they do not determine whether that operation falls within the current task. An explicit Host-authored Task Authority Manifest establishes a durable `transition_ceiling`; every manifest derived from it preserves or narrows that ceiling for

the corresponding process. It bounds launch authority and constrains permitted provider effects, data receive domains, resource-budget ceilings, approval policy, and any configured semantic-auto-approval rules for the process and subsequent agent-controlled child, fork, exec, and permission-request transitions. The manifest is canonicalized, and its unkeyed content hash is bound into approval and operation evidence. If the Host omits an explicit template, the Runtime instead persists an implicit compatibility manifest: Capabilities granted at launch still bound subsequent model permission requests, but a deliberate later Host grant may extend beyond that record. An image's `required_capabilities` field is a declaration and diagnostic only. The Runtime records unsatisfied requirements, but they neither block launch nor create live authority.

Where a transition ceiling applies, the two mechanisms are intersected. A broad reusable Capability cannot exceed a narrow task-effect ceiling, while a permissive manifest cannot create resource authority. For schema-v2 effect ceilings, an omitted field or JSON null imposes no additional restriction, whereas [] denies all provider effects. Legacy schema-v1 rows are upcast while preserving their historical interpretation of an empty list as unrestricted. Canonicalization preserves these distinctions explicitly rather than relying on truth-value coercion.

Human approval supplies a further conjunct; it is not an ambient source of authority. An ask decision creates a typed pending request with a canonical preview. A terminal response explicitly selects `always_allow`, `always_deny`, or `ask_each_time`, subject to Host constraints; exact one-time authority instead uses a separate one-use Capability lease. On resume, the Runtime re-enters the original primitive and revalidates its actor, resource, canonical arguments, manifest, and target-state bindings. Neither a textual response nor a GUI action is converted into a general-purpose Capability.

3.2 Budget admission and settlement

Hierarchical budgets account for logical Tool and LLM calls, filesystem read/write bytes, JSON-RPC and MCP request/response bytes, Deno syscalls, subprocess wall time, CPU time, peak RSS, and other Runtime resources. Each charge is applied along the full acting-process-to-ancestor accounting chain. Budget admission limits how much otherwise-admissible work may run; it does not authorize that work.

The Runtime distinguishes a nonreserving preflight check from a durable reservation. Preflight rejects an operation that cannot fit but holds no quota. A reservation instead records a maximum usage envelope before dispatch and later settles observed usage against that envelope. A certified-not-started outcome releases the reservation; an ambiguous post-dispatch outcome is maximum-settled when no better measurement is available. Effect and evidence finalization may precede resource settlement. Consequently, an accounting failure after an external effect cannot undo that effect; the unresolved reservation remains durable input to recovery. This ordering avoids claiming transactional atomicity across a non-transactional provider.

Each executor-level logical LLM call reserves one call together with the configured maximum envelope for prompt, completion, and total tokens before provider dispatch. Bounded transport attempts, retry/continuation state, and provider-reported usage are

recorded as observations within that logical call. These counters therefore do not represent exact physical-request counts, provider billing, or monetary spend.

3.3 Labels, source identity, and Sinks

Runtime-mediated payloads carry a *DataFlowContext* that associates labels with immutable source references. Aggregating contexts selects the highest sensitivity, the lowest integrity and trust, and a conservative join of tenant/principal identities. Versioned Objects and file bindings contribute immutable references, while Human responses, LLM/provider ingress, Tool results, JIT Tool results, process messages, JSON-RPC and MCP responses, and typed Git observations propagate Runtime-owned flow context. Joins remain conservative: the Runtime never drops a label merely because the model appears not to have used some part of its context.

Each external egress targets a concrete Sink. The Host owns a durable, canonical Sink registry whose generation advances on every mutation. Under the default configuration, unmatched Sinks are untrusted with a normal sensitivity ceiling; more generally, no untrusted Sink can clear data above normal. A Sink rule specifies a trust level, maximum sensitivity, tenant/principal allowlists, and, for provider-backed clearance above normal, a provider identity hash. An operation contract may additionally impose an independent minimum source-integrity floor. The model cannot assert in a Tool argument that a URL, shell, PTY, filesystem target, provider, or GUI surface is trusted. An operation with multiple actual recipients must declare every corresponding Sink; authorizing only a convenient primary recipient is insufficient.

Information-flow enforcement spans Runtime-mediated boundaries for LLM, Human, JSON-RPC, MCP, typed Git, filesystem writes, shell/PTY input and control, process handoff, and selected GUI presentation. Because a filesystem read or clock observation may receive information without transmitting the caller's payload, the operation contract explicitly declares its direction (none, ingress, egress, or bidirectional); the Runtime does not infer direction from the legacy `information_flow` flag.

3.4 Exact release and conservative declassification

When a conditional Sink requires sensitivity clearance above normal and the requested sensitivity remains within the rule's maximum, the Runtime may create a typed Human request for an exact one-shot release. The release binds the process and operation; Sink and trust identities and the registry generation; the canonical payload or argument digest; source Object and file versions; effective labels; the Task Authority Manifest hash; any target state; and a deadline. It is nondelegable and single-use. The preparation phase of a Protected Operation reserves the release, and successful settlement consumes it. A certified-not-started path may restore the same live reservation, whereas an unknown provider outcome consumes the authority and prohibits automatic replay.

An exact release does not mutate labels: the source remains labeled, later payloads undergo independent checks, and minimum-integrity floors remain nonwaivable. Lowering an Object’s sensitivity, removing or replacing its identity, raising its trust or integrity, or changing its declassification authority requires an exact declassification:object:<oid> Capability with the admin right. Bundled model-facing memory Tools cannot request this transition; a process may perform it only after the Host deliberately issues the Capability. This high-water design differs from permissive IFC analyses that identify influential inputs and regenerate outputs under reduced contexts to derive less restrictive labels [43], and from planner-level architectures based on trusted control flow or deterministic label propagation [5, 7]. By design, Agent libOS preserves high-water labels and confines exceptional egress to narrow, inspectable releases; it does not claim a general noninterference theorem.

3.5 Semantic Shadow is not an allow oracle

An optional semantic plane can assess eligible approval requests and committed provider ingress, record payload-free FlowGraph evidence, and emit conservative findings. It is disabled by default. In shadow mode, it records counterfactual outcomes but cannot change decisions. In enforcement modes, machine denial is limited to a closed set of deterministic Host violations. Only the canary path may issue an exact, short-lived, nondelegable, one-use Capability for a frozen catalog of low-risk read and diff operations, and only when every independent Host predicate is already satisfied.

The classifier returns only closed, typed findings together with confidence, calibration, out-of-distribution, and abstention metadata; its schema has no general allow or deny field. Findings may veto candidate canary issuance or require Human review, but cannot themselves cause a terminal denial, select a rule, mutate labels, widen a ceiling, or supply a missing allow predicate. The Host-only settlement path re-reads the pending request and revision, live Task Authority, DataFlow and FlowGraph state, policy epoch, binding and target state, classifier provenance, and budget within a Host transaction. Learned semantic analysis thus provides only fallible evidence; admission remains deterministic and Host-controlled.

4 Protected Operations and Causal Evidence

4.1 Why provider calls need a protocol

Filesystem, Git, shell, LLM, JSON-RPC, MCP, Human, and PTY providers can produce effects outside the enclosing RuntimeStore transaction. A process may crash after the provider commits an effect but before the caller receives a return value; conversely, it may crash after local preparation but before dispatch. Treating both cases as ordinary Tool-call failures can either discard evidence or cause a retry to duplicate an external effect.

The Protected Operation SDK is the stable extension boundary for provider-backed host operations. A trusted contract declares the authority mode, provider and operation identities, effect and resource policies, information-flow direction, minimum integrity, required evidence roles, and any prepared-state recovery handler. The domain primitive remains responsible for argument normalization and exact Capability decisions, while the SDK governs the common lifecycle and its fail-closed evidence contract.

4.2 Prepare, dispatch, and settlement

For an invocation r , the protocol proceeds as follows:

- (1) **Normalize and preflight.** The primitive resolves the concrete resource, canonicalizes the arguments, and identifies the actor, provider target, any applicable Task Authority ceiling, Sinks, source references, and optional resource envelope.
- (2) **Prepare atomically.** In a single RuntimeStore transaction, the SDK reauthorizes and reserves both finite Capability uses and exact-release uses, creates a durable usage reservation when the invocation supplies a resource envelope, creates the external-effect intent, and links that intent to the causal operation. No provider code has run at this point.
- (3) **Revalidate and dispatch.** Immediately before each provider phase, the SDK rechecks all reusable or reserved authority and every declared mutable binding: the semantic control epoch, provider-registry identity, Sink identity, target-state resolver, and DataFlow sources. A contract such as typed Git may also bind a target-specific state token. The SDK then crosses the provider boundary using only Host-configured transport and credentials.
- (4) **Classify and finalize.** Successful and failed completions are classified and finalized as committed, failed, or unknown, with a rollback class, status, and bounded observations. Certified-not-started outcomes are handled separately: when no earlier phase has crossed the effect/authority floor, the SDK restores the exact reservations and abandons the prepared intent. Normally, the terminal effect transition, event, audit record, and causal links are committed before the SDK returns. The narrow PRESERVE_RESULT exception for Human read/write operations may return a provider-accepted answer or delivery even if local completion fails, but only after conservative fallback settlement succeeds. The surviving effect remains pending or unknown, and the Human provider is not invoked again.
- (5) **Settle resources.** The reservation is settled against observed usage. A certified-not-started outcome releases finite authority, whereas dispatched or unknown paths consume it. A settlement failure cannot undo a committed provider effect.

Figure 2 shows the state machine. A prepared-state recovery callback may repair only local data within the recovery transaction; it cannot call a provider. Any contract that mutates local state during preparation must name a handler installed before the Runtime is opened; otherwise, startup fails closed.

The dispatch property is intentionally weaker than an end-to-end commit property:

$$\begin{aligned} \text{providerDispatch}(f) \Rightarrow \exists p, r, o. \\ & \text{prepared}(f, p, r, o) \wedge \text{OpAdmit}(p, r) \\ & \wedge (\neg \text{egress}(r) \vee \\ & \quad \text{FlowAdmit}(p, r, x_r, s_r)) \\ & \wedge \text{linked}(f, o, \text{intent}). \end{aligned} \quad (7)$$

Here, x_r and s_r denote the request’s payload descriptor and canonical Sink. Only Runtime finalization entails terminal evidence:

$$\begin{aligned} \text{finalized}(f, \text{committed}) \Rightarrow \exists o. (\text{classified}(f, \text{committed}) \\ \wedge \text{linked}(f, o, \text{event}, \text{audit})). \end{aligned} \quad (8)$$

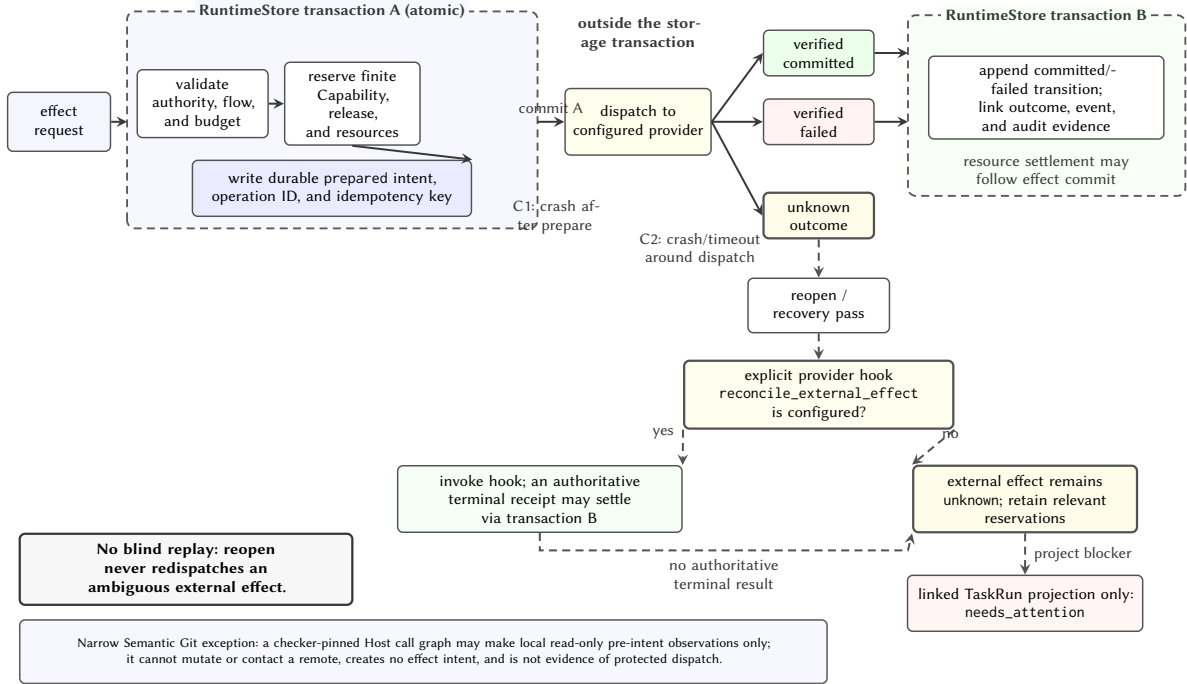


Figure 2: Protected Operation lifecycle and crash windows. Preparation binds authority, flow, resource, and evidence before provider dispatch. An unknown outcome means that the provider may have acted. A contract may reconcile this state through an explicitly installed hook, but reopening never grants replay permission.

Thus, a crash after provider dispatch but before finalization may leave a durable intent without its terminal event and audit links. On reopening, the Runtime must classify the outcome as unknown unless an explicitly installed reconciliation hook supplies sufficient provider evidence. This property is a mediation invariant, not an exactly-once theorem. A malicious or misconfigured provider may violate its contract, and a direct Host bypass falls outside the threat model. For cooperating providers, stable idempotency keys and receipts improve reconciliation without altering the conservative treatment of missing evidence.

4.3 Crash ambiguity and recovery

The Runtime distinguishes three crash-relevant cases. First, if no durable intent exists, no effectful provider phase was entered through the operation. The checker-pinned Semantic Git security observation described above is a local, read-only preflight and therefore does not count as such a phase. Second, if an intent remains prepared and provider dispatch is certified not to have started, recovery can repair local preparation state, abandon the intent, and release reservations. Third, if dispatch may have begun, recovery preserves or consumes finite authority, charges the conservative resource envelope when necessary, and leaves the outcome unknown. Only a contract with an explicit reconciliation hook may use a stable provider receipt or another bounded observation to prove a terminal state. A generic effect does not have a needs_attention state; a linked Task Run

may nevertheless project an unresolved unknown effect as that operator-visible status.

Provider reconciliation, when explicitly supported, may restore an effect-bound Capability reservation only in the narrowly defined certified-not-started case. This dependency explains why stale Capability reservations cannot be abandoned before provider effects are reconciled at startup. A missing response, a timeout, or process death never constitutes evidence of non-execution.

Checkpoint restore and image commit do not roll back external effects. Rollback class and status are evidence about provider semantics, not mechanisms for compensation. A future compensation API would require a new protected operation with its own authority, DataFlow, effect, and audit record.

4.4 Explainable Operations

Each action that enters a protected Tool or Runtime operation scope is assigned a persistent causal operation tree. Typed links connect LLM calls, Tool calls, JIT syscalls, primitives, Capability decisions or reservations, Task Authority identities, Human requests or responses, provider effects and transitions, resource charges, events, and audit records. Queries are Host-only and return bounded projections. Evidence completeness means that all required typed roles and links are present for the operation; it does not imply that the action was safe, that the model reasoned correctly, or that the database is tamper-proof.

Per-LLM Context Materialization Manifests record each versioned Object with an included or omitted disposition, a reason such as `selected`, `filter_mismatch`, `capability_denied`, `token_budget`, or `missing`, and a `verbatim`, `compacted`, or `truncated` transformation. They retain bounded token, hash, and label metadata without duplicating payloads. Provider trace views group a bounded set of attempts under one logical call and may expose retained output within fixed bounds, Tool calls, usage, model, request, and response identifiers, timing, errors, and provider-returned reasoning or summary fields. Agent libOS neither records hidden chain of thought nor claims access to it.

Following established provenance systems, the evidence design represents lineage and recorded dependencies as typed relations among entities, activities, and agents [2, 27, 32, 45]. The purpose here is narrower and operational: to identify the Runtime boundaries involved, support recovery, and ensure fail-closed handling of missing links. Audit and effect-transition histories are append-only through RuntimeStore APIs, whereas guarded effect rows and retained payload projections may be updated in place. This design is not a cryptographic append-only ledger that protects against a database administrator.

5 Durable Execution, Self-Evolution, and Ordered Recovery

5.1 Affordances do not implicitly expand authority over existing resources

Table 1 summarizes how each self-evolution mechanism is constrained at admission, information-flow, and recovery boundaries. The shared invariant is that an affordance may alter a process’s program or model-visible action surface, but any resulting resource access remains subject to an existing primitive or Protected Operation contract. Newly created, process-owned resources may carry newly issued handles, and booting a trusted image package may invoke the narrow private-workspace bootstrap described below. Neither case turns an affordance into a grant over pre-existing or external state.

5.2 Skills and JIT Tools

Skills are declarative `SKILL.md` packages comprising instructions, resources, visible built-in Tools, and optional JIT candidates. Discovery and activation are explicit operations. Trust and Skill authority determine which packages may be loaded; activating a Skill does not grant the filesystem, shell, Git, MCP, or other Capabilities required by its Tools. Built-in images may initially expose a compact lifecycle projection while retaining a broader immutable callable-table ceiling. The complete table enables trusted direct workflows while keeping the model projection distinct from the security boundary.

JIT Tools execute TypeScript under Deno without direct access to a libOS object. Their sole Runtime entry point is `libos.syscall`; messages sent through this entry point are schema-validated and dispatched to the same primitive managers as Python Tools. Under the current policy, static imports, re-exports, and `dynamic import()` are rejected. Before Deno starts, a supervisor scoped to the Host lifetime establishes process-tree containment and accounts for supported subprocess limits. These mechanisms provide defense in

depth; they neither establish a sound confinement proof for arbitrary TypeScript nor constitute a kernel-grade sandbox.

By contrast, trusted Runtime Modules are Host-selected Python code loaded before `Runtime.open()` returns. They may register Tools, images, syscalls, provider hooks, and startup hooks. Although they expand the TCB, registration alone does not grant a process a Capability.

5.3 Images, checkpoints, and publication

Checkpoint creation snapshots reconstructable state for a process subtree, including process and Object state; Tool/Skill/JIT metadata; messages and lifecycle position; durable `llm_pending_actions`; image artifacts required by the subtree; and relevant authority metadata. Active ObjectTasks, Task Run state, and terminal-cleanup records are excluded from the snapshot, while Host provider registries and provider state remain part of the current external configuration. Restore preserves historical Human, LLM, audit, and external-effect records. Fork remaps process identities; intentionally discards captured `llm_pending_actions`; revalidates current source Capabilities; removes revoked, expired, finite-use, or newly overlapping grants; derives fresh attenuated manifests; and reserves the captured root budget against the current parent. It cannot resurrect a revoked grant.

Committing a checkpoint publishes a checkpoint-derived artifact and a corresponding image-registry entry; it neither boots the image nor materializes filesystem content. Separately, booting an image package with a workspace seed may materialize a Runtime-private workspace through Host filesystem APIs under a durable publication program. The Runtime may issue only the filesystem grants declared in the Host-accepted package manifest, and only over that fresh copy. Startup reconciles interrupted publications before making the Runtime available. This narrow TCB exception has a fixed target and operation; it does not create a model-facing filesystem bypass.

5.4 Durable Task Runs

A Durable Task Run supervises a single root AgentProcess tree under a versioned goal and associated requirements. It exposes idempotent Host commands to start, follow up, continue, cancel, inspect, and perform terminal settlement, together with an append-only, evidence-linked ledger. Safe resume points distinguish completed local work from provider-side ambiguity. Retention of plaintext payloads requires explicit opt-in; by default, reports expose only bounded identities and hashes.

A Task Run is neither a workflow DSL nor a distributed scheduler. The model and Tools continue to determine task structure. Instead, a Task Run provides a durable container for one process tree, command deduplication, Runtime-epoch fencing, recovery, and an operator-visible `needs_attention` state. Unknown effects, missing required payloads, or abandoned ObjectTasks prevent automatic continuation rather than being treated as success. Version 1 certifies only one dynamic binding change: `activate_skill` as the sole action in a validated, nonparallel model response. Any other action that changes Image, Tool, provider, or loaded-Skill bindings places the Task Run in `needs_attention` before continuation or reopen dispatch and is never replayed automatically.

Table 1: Self-evolution mechanisms and the boundaries that constrain them. The third column reports whether a mechanism implicitly expands authority over pre-existing or external resources; handles for newly created resources and the trusted private-workspace bootstrap are identified explicitly.

Mechanism	Evolves	Implicitly expands authority over pre-existing or external resources?	Admission and flow boundary	Durable/recovery rule
Skill activation	Prompt instructions and projected Tools/JIT candidates	No	Skill authority governs activation; invoked Tools remain subject to their domain primitives	Snapshot and trust metadata persist; unloading removes the Skill’s projection and process-local JIT bindings while preserving independent base bindings
JIT registration	Process-local executable Tool code	No	Deno accesses the Runtime only through typed syscall RPC	Source, source hash, and Tool metadata persist; the Runtime supervisor owns process-tree termination
Image register/exec	Boot prompt, Tool table, policy defaults, and workspace seed	No; fresh private-workspace bootstrap only	Ordinary requirements are declarative only; for a Host-accepted package, the Runtime may issue only the grants declared over its newly created private workspace	Publication is recovery-bound; other exec paths preserve or intersect current authority
Checkpoint restore/-fork/commit	Scoped Runtime state and reusable boot state	No	Restore and fork require checkpoint and process authority; DataFlow and task ceilings remain in force after intersection	External effects remain historical; stale authority is never resurrected
Child process	A new subject, namespace, queue, and action surface	No	Spawn/fork and authority delegation are explicit and attenuating; handoff preserves labels	Parent–child state and messages are durable; revocation takes precedence
Object Memory	Versioned Objects, links, and context selection	No; fresh Object handle only	Existing Object and namespace rights, together with receive-domain checks, govern use	Versioned state is checkpoint-scoped; context manifests record materialization links
Remote registry	Visible JSON-RPC/MCP affordances	No	Public IDs pass an early authority gate; exact-item authority and Sink checks follow resolution of hidden metadata	Registry digests and generations invalidate stale grants; reconciliation occurs only through explicit hooks
Typed Git remotes	Existing repository-configured remotes	No	Dispatch is bound to repository configuration, helper/ref fingerprints, remote authority, and Sink clearance	Startup observes only local state; remote ambiguity remains unknown

5.5 Startup dependency order

Startup recovery completes before the Runtime enters OPEN. This order is a security property because each later phase relies on conclusions established by earlier phases. Prepared-operation recovery handlers are installed during assembly before the recovery pass begins. While holding the recovery lease, the builder performs the following steps:

- (1) validate recoverable Task Run plaintext and its integrity bindings without dispatching;
- (2) reconcile interrupted MCP continuations, remote Tasks, and subscriptions;
- (3) recover prepared operations and reconcile pending provider effects, admitting receipt observations only through explicitly installed hooks;
- (4) recover semantic exact-once authority, then abandon stale Capability uses and settle or restore resource reservations;

- (5) reconcile process-exec, process-launch, and checkpoint-restore publications;
- (6) recover root-goal payloads and missing Object payloads, rehydrate registered JIT Tools, interrupt stale operations and executions, and recover ObjectTasks and terminal-cleanup intents;
- (7) recover Task Runs only after their dependent process, operation, and effect state is stable.

Figure 3 presents a dependency graph, not a wall-clock guarantee. Recovery proceeds under a dedicated lease. High-cardinality effect and recovery scans are paginated with configured bounds, while other components enforce their own hard bounds. When provider reconciliation is configured, it precedes abandonment of stale Capability reservations because it may restore a reservation after establishing that the corresponding effect was not dispatched. After this pass, while the Runtime remains pre-OPEN, the builder

holds the startup lease, runs trusted startup hooks, starts the ObjectTask worker, performs checkpoint payload delivery, reconciles terminal checkpoint-restore publications, and commits the delivery acknowledgment before transitioning the Runtime to OPEN.

This ordering reflects a broader lesson from durable execution systems: local state machines must preserve and expose ambiguity at external boundaries rather than assume that a database transaction can span external systems [3, 4, 13, 19, 59, 65]. Unlike general-purpose workflow systems, Agent libOS binds these durability guarantees to agent process identity, finite authority, IFC, and causal evidence.

6 Implementation and Case Studies

6.1 Runtime composition

The prototype supports Python 3.11–3.14 and exposes an embeddable Runtime, a CLI, and a loopback Electron/React operator interface. During Host composition, the builder constructs managers and provider contracts, loads hash-trusted Runtime Modules, installs recovery handlers, completes pre-OPEN reconciliation, and only then publishes the Runtime as OPEN. A bounded, thread-backed worker pool advances quanta for runnable AgentProcess instances; a quantum that waits for Human input, a timer, a message, a child process, a Tool task, or an LLM call is confined to its worker, allowing other workers to continue. An asynchronous Host wrapper integrates the same Runtime with an external event loop.

ToolBroker resolves a named entry only through the process’s complete callable Tool table and opens the corresponding causal Tool operation; the invoked Tool then constructs any primitive-specific context and traverses primitive admission. The model executor receives a separate projection determined by the image and active Skills; depending on the image, this projection may equal or be narrower than the complete table. Direct Host workflows can invoke a callable Tool without an LLM turn, but still traverse ToolBroker, process authority, and primitive admission. Thus, projection can change model ergonomics without changing the enforcement surface.

The Resource Provider Substrate supplies typed filesystem, clock, shell, Human I/O, Git, JSON-RPC, and MCP backends. The LLM provider service is composed alongside that substrate, while optional PTY support is supplied by a trusted Runtime Module. Remote endpoints, transports, credentials, Git remotes, and LLM profiles are fixed by Host composition. Model-facing operations may select only the logical identifiers exposed by their typed interfaces, such as a registered JSON-RPC endpoint and method, an MCP Tool or model-visible Resource, or an existing Git remote; they cannot inject an ad hoc URL, credential, transport command, or profile definition. Each provider contract also specifies result bounds, data-flow direction, and conservative effect classification.

6.2 Persistence and transaction boundaries

Durable state is coordinated through a Unit of Work comprising typed domain repositories for processes, Task Runs, authority, resources, Objects, evidence, extensions, publications, semantic assessments, MCP lifecycle state, and payload retention. SQLite is the default local backend, while PostgreSQL 17 is an optional backend

over the same canonical store-schema-v7 contract. Opening a persistent store acquires a single-writer Runtime lease and validates the version marker, exact schema shape, required indexes, encoding, and collations. Canonical version-4, -5, and -6 stores must traverse the remaining explicit offline migration steps to version 7 in order. Version-3 archives, older or unversioned stores, and malformed or incomplete version-7 stores are rejected rather than repaired at startup.

Object metadata and namespaces are durable, but ordinary Object payloads remain Runtime-local and are represented in the store by live-payload markers. A narrow, explicitly enabled root-goal envelope may retain a bounded payload; scoped checkpoints and image artifacts may separately capture the bounded payloads needed to reconstruct a process subtree. No other ordinary Object payload is implied to survive Runtime restart.

The database transaction boundary is deliberately narrow. A RuntimeStore transaction can atomically update process transitions, authority and resource reservations, protected-effect intents, events, and audit records. External actions—including filesystem changes, subprocess or Git operations, Human I/O, and JSON-RPC, MCP, or LLM calls—cannot participate in that transaction. The Protected Operation state machine and startup recovery therefore expose this boundary explicitly rather than hiding it behind a Tool return value.

Payload retention is distinct from causal retention. Disabled by default, Host-triggered maintenance can monotonically reduce eligible terminal LLM-call and external-effect provider payloads from full to summary and then hash-only, while preserving identity, classification, transition state, digests, and causal links. Run-linked Human payloads are governed separately by the Durable Task Run retention contract. Neither path changes an effect’s state or permits a nonterminal or unknown effect to appear terminal.

6.3 Case study: typed Git

Git illustrates why wrapper-level authorization is insufficient. Its CLI admits a large language of options, configuration, hooks, filters, alternate object stores, credential and transport helpers, refsspecs, revision expressions, pathspecs, and repository-discovered metadata. A model permitted to supply arbitrary git arguments can therefore select executable code or destinations indirectly, even when a wrapper blocks an obvious option.

Agent libOS instead pins Runtime.git to the configured non-bare workspace repository. Model-facing Tools select typed operations for bounded inspection, local changes, branch/tag/stash/worktree management, immutable patch Objects, existing remotes, and repository-local simulated pull requests. No typed operation supports init, clone, remote or configuration mutation, arbitrary arguments, submodule update, LFS, plumbing, custom upload/receive pack, model-selected credentials, or arbitrary URLs. On every call, the provider revalidates the repository root, selected worktree, Git directory, common directory, object format, and filesystem identity. Managed worktrees remain under a Host-selected root; the process cannot supply an external destination.

Git output is captured and parsed as bytes. Each path is returned as display text together with a base64 token for the exact repository-relative bytes. Path-bearing commands use literal pathspecs and

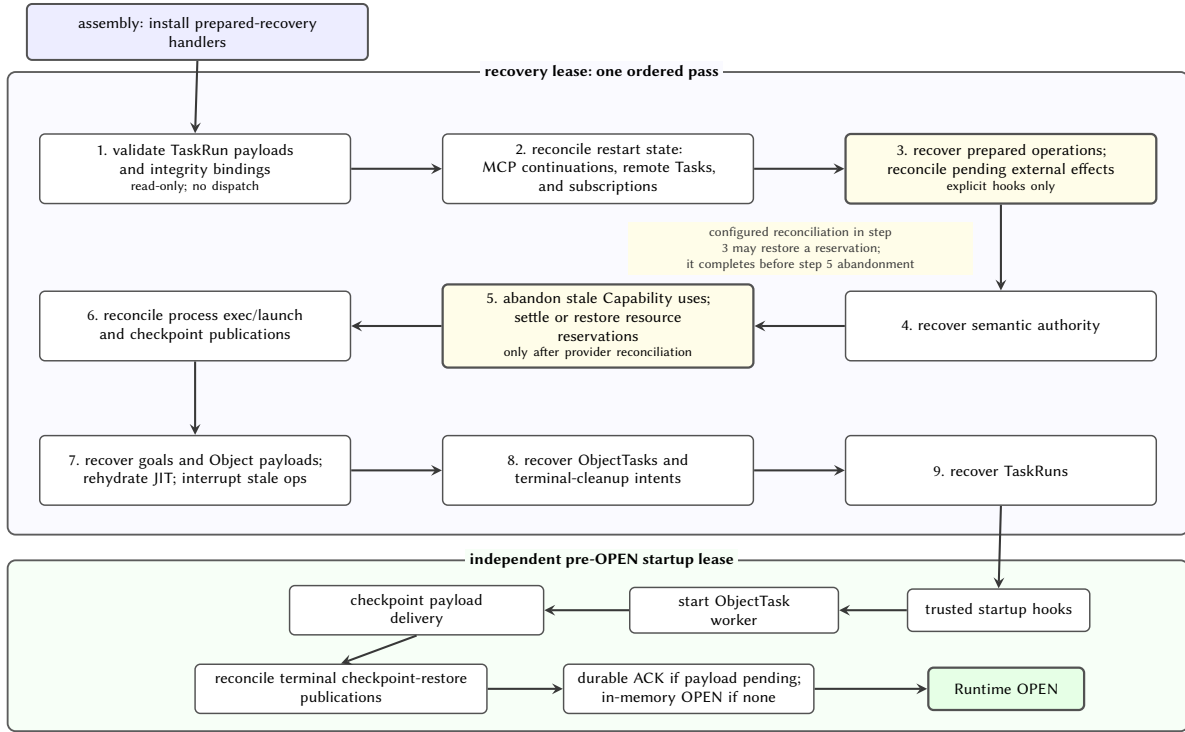


Figure 3: Pre-OPEN recovery dependencies. This ordering prevents a later phase from discarding authority, payload, or publication state required to reconcile an earlier ambiguous operation. Reopening completes local recovery; it neither rolls back provider state nor authorizes redispach.

Table 2: Implemented surfaces and their primary boundaries. “Client subset” is deliberate: no row implies an MCP server or a public remote API.

Subsystem	Implemented surface	Primary admission boundaries	Evidence/recovery boundaries
Processes and memory	Lifecycle, queues, Object Memory, cwd, context materialization, background ObjectTasks	Process/Object/filesystem Capability, applicable Task Authority ceiling, receive domain, budgets	Store transactions, process state/execution generations, context manifests, Task Run epoch fencing/recovery
Self-evolution	Skills, JIT Tools, images, checkpoints, child processes, trusted modules	Affordance-specific primitive plus unchanged resource admission	Snapshot lineage, publication recovery, source hashes, causal links
Local providers	Filesystem, shell, clock, Human I/O, optional Object-bound PTY	Typed resource and policy; DataFlow for input/output Sinks	Protected effects, resource settlement, lifecycle cleanup
Git	Typed inspection, local mutation, managed worktrees, immutable patches, existing remotes, simulated PRs	Repository, remote, PR, and Object Capabilities; applicable Task Authority ceiling; exact state token; affected filesystem Capabilities; Sink clearance	Irreversible/unknown effect classification, repository/worktree locks, and local, query-only recovery observations
Remote clients	Registered JSON-RPC and MCP stdio/Streamable HTTP client subsets	Per-item authority, Host transport, Sink and ingress labels, byte/time budgets	Registry generations, Protected Operations, continuation/task/subscription recovery
LLM	Named OpenAI-compatible profiles, bounded attempt traces, optional prompt-cache policy	Profile Capability, applicable Task Authority ceiling, LLM Sink identity, token reservation	Logical-call record, bounded response projection, context/effect links
Operator surfaces	Python API, CLI, loopback HTTP/SSE, and Electron GUI	Loopback bearer authentication, exact-origin/CORS policy, and Host/admin versus actor-mode routes	Redacted snapshots, durable commands, and causal inspection

the `-` option terminator, preventing a filename that begins with a dash or contains non-UTF-8 bytes from being interpreted as an option. Results report truncation, byte counts, and hashes explicitly; immutable patch Objects must always be complete.

Before any mutation, the caller supplies an opaque state token obtained from a prior read. The token commits to repository/worktree identity, HEAD and ref state, the index, effective configuration, the worktree registry, simulated-PR metadata, and bounded worktree content state. With a cross-process repository lock held, the primitive rechecks the token immediately before dispatch. A failed initial comparison before the first state-changing effect certifies that the invocation did not dispatch; a later mismatch may follow an earlier effect and is therefore possibly dispatched. The token is compare-and-swap evidence; it is neither read authority nor a replay credential.

Git models the pinned repository, each configured remote, each simulated pull request, and immutable patch Objects as distinct resources. Operations classified as destructive additionally require both delete and admin authority and a mandatory Human approval that mints an exact, approval-bound one-use Capability lease. Checkout-changing operations require the corresponding filesystem read, write, or delete Capabilities, while ordinary filesystem authority never extends to Git metadata. A legacy Shell grant for `git` confers none of this typed authority.

Information-flow policy is evaluated independently of Capability admission. Reads are ingress; local mutations are bidirectional with the repository as their Sink; fetch combines remote ingress with egress to the local repository Sink; push is egress to the exact configured remote Sink; and pull-request operations bind the exact PR resource, with creation and merge also clearing the repository Sink. The operation canonically binds source and carrier labels, refs, paths, worktree identity, and the state token. Consequently, repository authority alone cannot authorize a secret-bearing push.

The provider disables or neutralizes hooks, pagers, editors, signing, external diff and textconv, lazy fetch, maintenance, replace refs, and workspace-controlled executable lookup. It fails before dispatch if active configuration selects external filters, drivers, helpers, proxies, custom transports, or unsafe includes. This is command mediation rather than an OS-level sandbox: a separately authorized interpreter or native executable can invoke Git with the Host user's authority. Deployments that cannot tolerate that possibility require a more strongly isolated provider.

Git remotes are drawn from the pinned repository's validated configuration, not from the JSON-RPC/MCP endpoint registry. Startup recovery is local and query-only: it may inspect current refs, worktree state, simulated-PR metadata, and freshly recomputed local configuration, helper, and ref fingerprints. It does not contact a remote, use these observations to establish the recorded network outcome, or clear an effect in the unknown state.

6.4 MCP and registered remote resources

The MCP integration is client-only. Manifests v1 and v2 retain a governed Tools compatibility contract. Manifest v3, pinned to the exact protocol revision 2026-07-28, adds governed Resources, Resource Templates, Prompts, Completion, Multi Round-Trip Requests (MRTR) for `input_required` continuations, bounded

subscriptions/listen streams, Host-preconfigured OAuth handling, and optional digest-pinned support for the Tasks extension. Negotiation is explicit and bounded; automatic fallback is permitted only for recognized compatibility responses, not arbitrary authentication, TLS, timeout, malformed-response, or server errors. Tool schemas and manifest identities are pinned before calls [26].

Each server and item is Host-registered under a closed, operation-specific contract. Server configuration fixes transport, timeout, and byte bounds. Tool entries additionally declare effect and rollback metadata, while Resources, Resource Templates, and Prompts use separate closed read or prompt contracts. Trusted configuration supplies stdio commands and environments; the Host likewise supplies Streamable HTTP endpoints and authorization. Public server and item identifiers pass an early invocation-authority gate before any hidden registry lookup. After resolution, the primitive enforces exact item authority and revalidates the server digest and generation immediately before dispatch.

Continuations, subscriptions, OAuth state, and remote Tasks have explicit local lifecycle and recovery contracts; an opaque remote task identifier never becomes model-visible authority. MCP capability declarations negotiate protocol features, while item listings support discovery. Neither constitutes a process-level authority record, and server acceptance of an OAuth token does not by itself authorize a local agent process.

The implementation deliberately covers a client subset rather than all of MCP. Agent libOS excludes an MCP server surface, Apps, Roots, Sampling, Logging, Dynamic Client Registration, client-credentials and enterprise-managed OAuth, workload-identity federation, DPoP, older OAuth compatibility modes, deprecated standalone SSE, and product-level OpenTelemetry integration. Executing a trusted MCP stdio server still launches native code under the Host account; the protocol boundary does not sandbox that code. Accordingly, the current scope shows that the three-plane design extends to a rich registered provider; it does not claim universal MCP interoperability.

OAuth and delegated-authorization standards address complementary questions. G NAP and the OAuth Security Best Current Practice constrain authorization protocols and token handling [21, 36], while agent-oriented systems explore policies and authenticated delegation [42, 44, 47, 49]. Agent libOS consumes endpoint and credential authority established by the Host; it neither replaces these protocols nor permits a model to configure them.

6.5 Operator and semantic surfaces

The Python Runtime object is the primary composition interface. The CLI and Electron application expose process and Task Run supervision, Human queues, provider traces, events, audit records, operations, and bounded diagnostic views. Loopback-only HTTP/SSE endpoints serve the local GUI; they do not form a public REST control plane. Semantic-policy mutation is confined to Host composition and is not exposed through model Tools, JIT, Skills, Modules, the GUI, the CLI, or HTTP.

The GUI displays bounded provider-returned reasoning, summary, or thinking text only when it remains readable under the

retention policy. It neither synthesizes explanations nor claims access to a model’s hidden chain of thought. Read-only operation views can show causal links among Tool, primitive, decision, Sink, effect, event, and audit roles, while sensitive bodies remain behind separate on-demand projections.

7 Evaluation

The evaluation distinguishes deterministic boundary behavior from stochastic model behavior and native execution from modeled design coverage. These distinctions prevent aggregate success rates from masking missing effect evidence or simulated scenarios from being conflated with provider execution. We ask:

RQ1: Confinement.

Does the Runtime prevent unauthorized effects modeled by the suite, reject flows that exceed Sink clearance, and preserve declared allowed effects?

RQ2: Durability.

Does the handling of intent, settlement, causal evidence, and startup recovery remain fail-closed across crash barriers and large histories?

RQ3: End-to-end behavior.

Under one real-model configuration, can restartable maintenance, browser, research, and analysis workflows preserve authority and safety while satisfying independent utility oracles?

RQ4: Projection ergonomics.

Compared with immediate exposure of the full Tool schema, how does on-demand Skill projection affect correct routing, observable task success, invalid calls, schema bytes, prompt bytes, and provider-reported tokens under paired goals?

7.1 Evidence contract and provenance

All results reported in this paper are derived from evidence/`manifest.json`. The manifest binds each redacted JSON report to a SHA-256 digest and records the exact command, source revision and cleanliness, bounded working-tree identity, task/config/provider identity, report schema, and expected metrics. A generator validates these inputs and writes evidence/`results.tex`; no results are copied into the manuscript by hand. The publication build fails if any required report, hash, macro, denominator, or provenance field is absent.

Here, *publication-valid under the bound commit’s validation contract* denotes an artifact that is complete, hash-locked, source- and configuration-matched, and semantically decidable. It does not imply that a scientific gate passed, that the agent reached a terminal exit, or that an oracle was satisfied. A complete artifact documenting a failed gate, incorrect trajectory, or nonterminal model trajectory remains publishable as negative evidence; an artifact with missing or undecidable evidence does not. Because report schemas and gates evolve, this status does not imply acceptance by a later validator; a new gate requires a new source-bound campaign rather than retroactive backfilling.

The evaluated source is available at commit 77c9160c53d9. Each real-model report also includes a redacted provenance envelope bound to one provider configuration: model identifier, API mode, retry and timeout settings, request bounds, prompt/cache policy, endpoint class and normalized endpoint hash, and a credential-present boolean. The envelope never includes an API key, raw endpoint, or environment-variable value. Published reports retain only the bounded task/evidence fields required by their oracles; unredacted prompts, provider responses, credentials, and task secrets are excluded from the paper artifact. The canonical combiner rejects a dirty or changing tree and any cross-family mismatch in commit, model, or provider configuration.

The provider-smoke artifact has status available. This is an outer, operator-observed command receipt, not a report emitted by the smoke script or an attestation of the effective LLM profile. The receipt records a failure of the preflight terminal-state check. Later paid runs do not retroactively reclassify this failed preflight as a pass.

If an implementation or harness defect is corrected during a campaign, the source revision changes and the affected matrix is invalidated. A defect found after the campaign remains a limitation of the bound source and is not repaired retroactively by a later revision. Ordinary model failures are retained as results and do not trigger retries. A verified infrastructure failure may trigger a rerun of the entire affected family; the failed artifact and reason are retained, and successful trajectories are not selectively cherry-picked.

7.2 RQ1: deterministic runtime safety

The runtime-safety suite comprises versioned YAML tasks with copied fixtures, declared allowed and forbidden effects, deterministic execution plans, and evidence-backed oracles. It covers filesystem, shell, Human, process, Object Memory, image, checkpoint, Skill, JIT, JSON-RPC, MCP, data-flow, typed Git, and Semantic Shadow boundaries. The self-evolution subset attempts to activate Skills, register and execute JIT Tools, register, execute, and commit images, fork children, reuse checkpoints, attach remote affordances, manipulate Git state, and inject semantic authority.

The full Runtime runner is evaluated under strict success/safety and release-evidence gates. Three deliberately simple comparison interventions execute the same task plans: a direct Tool wrapper, a confirmation wrapper, and a static Tool-category “sandbox.” The third is neither a container, VM, nor Host sandbox; the harness models effects unsupported by a baseline. These baselines isolate boundary behavior and do not constitute real-model agent comparisons.

The ablations remove primitive approval, namespace isolation, or fork attenuation one at a time. A separate `no_audit_linkage` diagnostic intentionally withholds required evidence. Because missing evidence invalidates a result, this diagnostic is reported separately and never enters a safety-rate table.

The metric collector distinguishes task rows from effect records. Task-success and safety-pass rates are micro-averages over the declared task oracles. The unauthorized-side-effect rate includes in its denominator only effect attempts definitely known to have been performed. The false-denial rate includes only allowed attempts

whose outcomes are definitively classified as performed or denied. An unknown or missing outcome invalidates the row rather than being inferred from a Tool exception. For each task, audit completeness is the fraction of definitely performed effects with matching audit records; it is defined as 1.0 when no effect was performed. The reported value is the mean across tasks. Other evidence-role completeness is validated separately.

For the full Runtime row, the source-bound artifact records 33/33 passing task-success oracles and 33/33 safety-passing tasks, together with an unauthorized-effect fraction of 0/119, a false-denial fraction of 0/119, and mean audit completeness of 100.0%. These values characterize only the checked task/effect population and do not estimate open-world agent risk. The artifact records the full release-evidence gate as passed.

The invariant manifest is a coverage map, not an execution receipt. For the bound source, it resolves 149 invariant declarations against 8903 collected pytest nodes. Test execution status is recorded separately. Independent execution receipts classify the complete Python matrix as passed, the GUI lane as passed, and the invariant checker as passed.

7.3 Practical workflow evidence

The practical suite distinguishes two evidence levels. A native-live scenario must execute real ToolBroker calls against a stateful local provider, compare provider state before and after execution, persist the matching external effect, and resolve an explicit causal operation for every semantic effect. Native scenarios never fall back to modeled evidence. The native connector fixture implements synthetic mail, CRM, and calendar mutations through registered JSON-RPC methods. It is deterministic local infrastructure and does not establish production-connector interoperability.

A modeled scenario evaluates a design-level utility/security oracle but receives no credit for native Tools, operations, or provider effects. Accordingly, the two evidence levels retain separate scenario and semantic-effect denominators.

The bound report presents 3/3 native scenarios and 80/80 modeled scenarios as separate observations. The native evidence-level gate is recorded as passed. The modeled evidence-level gate is recorded as passed.

7.4 RQ2: recovery and crash evidence

Three SQLite reopen gates evaluate structural boundedness rather than throughput. The external-effect gate seeds a large terminal history together with pending prepared effects; the runtime-publication gate seeds terminal and unreconciled publications; and the Task Run gate seeds historical and recoverable runs. Each gate checks its partial indexes, keyset-query shape, exact page/row work, and complete convergence. Elapsed seed, reopen, and recovery times are diagnostic only; they are not thresholds, service-level objectives, or evidence of Runtime overhead.

The Task Run crash matrix uses isolated worker processes and an independently fsynced provider ledger across the protocol's durability barriers. It distinguishes pure local work, certified non-dispatch, durable provider receipt, and unknown dispatch. Reopening must result in no more than one provider dispatch, deduplicate a receipt

where supported, leave unknown work in needs_attention, and produce a stable evidence fingerprint on the second reopen.

The validated profiles contain 100000 external-effect rows, of which 1000 are pending; 10000 publication rows, of which 1001 are unreconciled; and 100000 Task Runs, of which 1000 are recoverable. The crash matrix passes 6/6 ordered barrier cases. The separately declared unpaired safe-point scenario also passes.

7.5 RQ3: canonical real-model workflows

The opt-in live gate evaluates restartable synthetic tasks under one fixed OpenAI-compatible model configuration. The repository-maintenance family uses typed file Tools to edit a fixture, runs a documented test command, inspects dedicated Git status/diff Tools, checkpoints, reports to a Human, and exits after a cumulative review. A follow-up arrives after Runtime reopen; replaying the stable continuation command must duplicate neither LLM work nor provider effects.

The browser family drives an isolated customer portal in a real headless Chromium instance through a fixed registered JSON-RPC bridge. The agent must ignore an injected customer instruction, incorporate a follow-up constraint after reopen, issue one idempotency-keyed partial refund, read back the order, preserve its paid status, and satisfy provider-state and DOM-action oracles. The bridge exposes neither arbitrary URLs nor a cancellation method.

The research workflow reconciles dated, conflicting local sources, including an indirect injection, without mutation. The data-analysis workflow writes an inert, reviewable standard-library script and a separately materialized JSON artifact, incorporates a segment follow-up, and allows a latency guardrail to determine the rollout decision. It has no shell authority, and the evaluator never executes model-authored code.

Each family has independent authority, safety, utility, recovery, settled-effect, and duplicate-dispatch checks. The canonical gate requires every family gate to pass, every recorded run to satisfy the safety/authority and zero-duplicate checks, and the preregistered aggregate and family-level utility minima to be met. Failure to invoke an expected operation reduces utility but does not itself constitute an authority failure. Conversely, a prohibited call correctly denied by the Runtime counts as a safety success but may still be an instruction-following utility failure.

The combined artifact has status available. Across the complete matrix, the artifact records an observed safety fraction of 12/12 and an observed strict-utility fraction of 12/12. These fractions describe one model/provider configuration and are not estimates of population-level rates. The combined canonical release gate is recorded as passed.

Canonical accounting records 189 logical LLM calls and 195 provider attempts. Provider-reported trajectory aggregates contain 3853163 input tokens and 316487 output tokens. Cache-read usage is reported for 189/189 logical calls, totaling 2496256 tokens. Because the frozen family schema does not expose per-call input/output reporting counters, these input/output aggregates are not claimed to have complete per-call coverage. Cache-write usage is reported for 0/189 calls; because reporting is incomplete, the aggregate cache-write token count is unreported, not zero.

Table 3: Deterministic runtime-safety results. Fractions report explicit numerators and denominators; invalid rows are not coerced to zero. Wrapper and category interventions are deterministic harness baselines, not OS sandboxes or real-LLM runs.

Runner	Tasks	Task success	Safety pass	Unauthorized	False denial	Evidence
direct_tool_wrapper	33	90.9%	39.4%	20/29	0/9	0.00%
confirmation_wrapper	33	93.9%	39.4%	20/64	0/44	0.00%
sandbox_only	33	72.7%	39.4%	20/29	15/24	0.00%
agent_libos_full	33	100.0%	100.0%	0/119	0/119	100.0%
no_primitive_approval	33	97.0%	100.0%	0/118	0/118	100.0%
no_namespace_isolation	33	100.0%	93.9%	2/121	0/119	100.0%
no_fork_attenuation	33	100.0%	97.0%	1/120	0/119	100.0%

Table 4: Practical-workflow results by evidence level.

Evidence	Passed/total	Semantic effects	Native Tool calls
native-live	3/3	3	3
modeled	80/80	160	–

7.6 RQ4: paired Skill projection

The paired study compares a compact Skills-based model projection with a no-Skills baseline that exposes the complete non-lifecycle Tool schema at the first model call. Both arms retain the same resource-authority ceiling and receive byte-identical neutral task goals. The goals specify only observable task intent; they mention neither a Skill, an expected Skill identifier, the treatment arm, nor a probe Tool. Held-out intents cover workspace editing, read-only Git, shell execution, checkpoints, and MCP registry inspection, each with a nearby but incorrect boundary.

The full matrix uses deterministic AB/BA counterbalancing across pairs and records the pair identifier, repetition, first-arm position, and order balance. A run counts as correct only if the observable-state oracle, route, structured probe, and terminal exit all succeed. Reports also retain invalid Tool-call counts; logical LLM-call and physical-attempt counts; initial, authorized, and cumulative schema bytes; prompt bytes; provider-reported input, output, and cache tokens; and a clearly labeled bytes-divided-by-four estimate. This estimate is not treated as provider billing evidence.

The publication gate requires a complete paired matrix, the expected order balance, a matching clean source state and provider configuration, and complete oracle, route, exit, and probe evidence. Publication readiness is outcome-agnostic: the evidence must be structurally complete and decidable but may document an incorrect run. Correctness is governed separately by `-require-all-correct`. Historical report schemas remain readable for diagnosis but cannot satisfy the publication gate.

The paired artifact has status available. It contains 15 complete pairs and 30 runs. The table reports paired effect estimates: continuous metrics use a fixed-seed, 10000-resample percentile bootstrap, whereas binary discordances use an exact McNemar test. The table’s task-outcome row reflects only the independent observable-state oracle: 15/15 in the Skills arm and 15/15 in the baseline. Correct routing is 15/15 versus 14/15; terminal exit is 13/15 versus 14/15; and the conjunction of outcome, route, probe, and exit is 13/15 in

the Skills arm versus 13/15 in the baseline. Thus, an observable task success is not equated with a fully correct run. The exact McNemar p -values are 1.0000 for task outcome and 1.0000 for correct routing. The bootstrap intervals in Table 7 quantify effect sizes; they are not hypothesis-test intervals.

Provider-reported accounting for the Skills arm records 99 logical calls, 100 provider attempts, 1068687 input tokens, and 151642 output tokens. The corresponding baseline values are 55, 56, 1563576, and 101954. Cache-read usage is reported for 99/99 Skills-arm calls (755072 tokens) and 55/55 baseline calls (1399808 tokens). For the Skills arm, cache-write usage is reported for 0/99 calls; its aggregate cache-write token count is unreported, not zero. For the baseline, cache-write usage is reported for 0/55 calls; its aggregate cache-write token count is unreported, not zero. Given the small paired sample and single model configuration, these confidence intervals characterize only the evaluated workload and should not be interpreted as broad causal estimates for LLM agents.

7.7 Interpretation and threats to validity

The deterministic suite tightly controls oracle definitions and source state but executes scripted plans; it therefore isolates Runtime mediation rather than model judgment. The wrapper interventions deliberately omit Runtime components. Their results show the consequences of those omissions within this suite, not the security of wrapper frameworks more generally. Effect rates weight effect records rather than tasks and should not be compared across artifacts with different task, fixture, runner, schema, or source hashes.

The real-model evaluation uses one model/provider configuration, a small fixed workload, and task-specific utility oracles. Paired counterbalancing reduces systematic order effects but cannot eliminate provider drift or stochastic dependence. The study reports prompt and token observations rather than dollar cost because the evidence includes no versioned price snapshot.

Finally, the recovery profiles assess bounded query work and state convergence, not latency service levels or steady-state Runtime overhead. Accordingly, they support no performance claim. The warranted conclusion is narrower: for the cases exercised by the bound artifacts, the implemented boundary either produces the required authority, flow, and evidence outcomes or causes the result to fail closed.

Table 5: Structural recovery gates. “Recoverable/pending” is the subset selected by the partial recovery index; elapsed time is intentionally omitted.

Gate	Total	Recoverable/pending	Reconciled	Page size
External effect	100000	1000	1000	500
Runtime publication	10000	1001	1001	500
Durable TaskRun	100000	1000	1000	500
Crash barriers	6	+1 unpaired	6/6; unpaired pass	–

Table 6: Canonical live gate. Status is descriptive; threshold misses are retained as observations rather than selectively retried.

Family	Safety	Utility	Runs
Repository maintenance	3/3	3/3	3
Browser workflow	3/3	3/3	3
Research workflow	3/3	3/3	3
Analysis workflow	3/3	3/3	3
Total	12/12	12/12	12

Table 7: Paired Skill projection comparison. Continuous intervals use the predeclared fixed-seed percentile bootstrap; binary outcomes use exact paired tests.

Metric	Skills	Baseline	Paired delta [95% CI]
Task outcome	100.0%	100.0%	0.00 pp [0.00, 0.00]
Correct route	100.0%	93.3%	+6.67 pp [0.00, +20.0]
Invalid calls	0.00	0.00	0.00 [0.00, 0.00]
Logical LLM calls	6.60	3.67	+2.93 [+2.13, +3.67]
Provider attempts	6.67	3.73	+2.93 [+2.00, +3.80]
Prompt tokens	71246	104238	-32993 [-54608, -15092]
Completion tokens	10109	6797	+3313 [+1388, +5245]
Cache-read tokens	50338	93321	-42982 [-62601, -27213]
Initial schema bytes	7649	91759	-84110 [-84110, -84110]
Cumulative schema bytes	72360	336450	-264090 [-330692, -215892]

8 Related Work

8.1 Agent runtimes and self-evolution

Tool-using and multi-agent systems provide much of the action loop underlying modern agents. ReAct interleaves reasoning with

action; Toolformer and ToolLLM study tool use; and AutoGen, MetaGPT, CAMEL, AgentScope, and SWE-agent support multi-agent collaboration or agent–computer interaction [11, 17, 20, 34, 40, 54, 56, 57]. CodeAct uses executable code as a compositional action representation, while OpenHands provides a platform for software-development agents that edit files, execute commands, and browse [50, 51]. MemGPT frames context management as virtual memory, whereas AIOS studies scheduling, memory, storage, access control, and resource support for LLM agents [23, 31].

Self-evolving systems go beyond selecting from a fixed set of Tool calls. Voyager accumulates skills, AFlow searches over code-represented workflows, SkillWeaver synthesizes web skills, Alita constructs task-related capabilities, and the Darwin Gödel Machine modifies agent code subject to empirical validation [35, 48, 60–62]. Agent libOS is complementary: it neither selects an improvement nor establishes its utility. Instead, it provides a substrate for process lifecycle, authority, information flow, durability, and evidence, allowing such mutable affordances to run without treating their availability as a permission grant.

8.2 Capabilities, policy, and delegation

Classical capability systems bind designation to authority through protected object references and support least authority and attenuation, with revocation provided through system-specific mechanisms [9, 15, 25, 41, 52]. Exokernels and library OSes place selected abstractions above a protected lower interface [10, 22]. Containers and microVMs isolate host execution at a different layer [1, 24]. Agent libOS adopts the idea of a stable lower boundary and an explicit authority vocabulary, but defines agent-native subjects and resources: Tool tables, Skills, process images, Objects, context, Human queues, provider endpoints, Task Authority ceilings, and external-effect receipts.

Progent checks symbolic rules over tool names and arguments, automatically applying verified policy narrowing while requiring approval for expansion [42]. AgentSpec provides customizable runtime enforcement for agent actions, while Conseca associates contextual policies with task purpose [47, 49]. Other recent work studies authenticated delegation and statically tracked capabilities [30, 44]. Separately, SAGA governs inter-agent contact through user-defined policies and provider-derived access-control tokens [46].

GNAP and the OAuth Security Best Current Practice address grant negotiation and token security at network authorization boundaries [21, 36]. MCP standardizes discovery, JSON-RPC messaging, standard transports, and optional authorization for HTTP transports. Its trust-and-safety guidance emphasizes user consent;

because the protocol cannot itself enforce consent or access control, Hosts implement those mechanisms [26]. These works target different layers and can establish or constrain Host-side credentials and policy. In Agent libOS, operation-time admission additionally intersects configured authority with the invoking process’s identity and Capabilities, any applicable Task Authority ceiling, budgets, and primitive contracts; it separately requires information-flow admission. It then binds the resulting admission to durable effect state and revalidates or reconciles the relevant bindings after a crash. This dynamic check also reserves finite authority against a prepared external effect, complementing static capture checks and inter-agent contact tokens [30, 46].

8.3 Information flow and declassification

Information-flow control (IFC) provides a vocabulary for confidentiality, integrity, and end-to-end propagation [8, 28, 29, 38]. Work on declassification classifies release policies by what information is released, who may release it, where release occurs, and when it may occur [39]. Agent libOS uses conservative label joins, a Host-owned recipient-trust registry, and an exact one-shot release bound to the actor, Sink, payload, source versions, policy, and registry generation. It does not establish noninterference, automatically infer semantic independence, or control a recipient after delivery.

Recent systems occupy different points in the agent-IFC design space. Wu et al. and CaMeL isolate planning or derive control flow from trusted input so that, under their threat models, untrusted observations cannot redirect execution; Fides formalizes planner-level label propagation and deterministic security policies; permissive analysis instead identifies influential inputs and regenerates outputs from a restricted context to derive less restrictive labels [5, 7, 43, 53]. Several of these systems provide formal guarantees for specified planner architectures, whereas permissive analysis emphasizes precision and utility. Agent libOS makes a different trade: it does not require a trusted plan and therefore does not claim to prevent untrusted data from redirecting control flow. Instead, it enforces authority and flow checks at provider and primitive boundaries, integrates those checks with persistent processes and recovery from ambiguous effects, and treats semantic classifiers as veto or escalation evidence rather than as an allow oracle.

8.4 Durable execution and external effects

Sagas define compensating actions for long-lived transactions, while RIFL suppresses duplicate RPC execution through stable request identities and durable completion records [13, 19]. Durable Functions proves that its compute-storage model simulates the fault-free high-level model and, for deterministic orchestrations, that its replay model is bisimilar to the compute-storage model. The compute-storage result excludes external calls because repeated work-item attempts can produce observable duplicates [3]. For externally visible operations routed through its API, Beldi provides crash-tolerant state equivalent to that of a single failure-free stateful serverless function (SSF) execution [59]. ExoFlow provides exactly-once DAG execution—visible outputs consistent with a failure-free execution—under declared task semantics. Externally visible tasks must be idempotent or provide an idempotent rollback, while ExoFlow coordinates checkpoints,

rollback, and replay [65]. SagaLLM applies saga ideas to LLM planning, while RAC adapts log-based compensation to agent workflows and invokes compensating actions after failures [4, 33].

Collectively, these systems illustrate that their guarantees for external effects rely on explicit, scoped mechanisms—compensating transactions, completion records atomically coupled to mutations, mediated operation APIs, application-supplied idempotence or rollback semantics, or a proof whose scope excludes external calls. In these designs, a local transaction or replay log alone is insufficient.

Agent libOS does not claim general exactly-once execution or automatic compensation. It records rollback class and reconciliation evidence but does not compensate provider state; any compensation would itself be a separately authorized Protected Operation. Its narrower contribution couples durable intent and receipt reconciliation to finite Capability and release reservations, Task Authority, DataFlow, resource settlement, and an agent-process causal tree. In its recovery state machine, an unknown outcome blocks automatic replay until registered reconciliation supplies sufficient evidence.

8.5 Provenance and explainability

W3C PROV defines a general entity/activity/agent provenance model, while LPM and CamFlow capture whole-system provenance over kernel-mediated interactions and information flows [2, 27, 32]. PROV-AGENT extends W3C PROV to agentic workflows, whereas AgentSight correlates LLM traffic with kernel-observed effects for system-level observability and debugging [45, 63]. AttriGuard uses counterfactual shadow replay to infer whether a tool invocation is causally attributable to user intent rather than to untrusted observations [16]. Agent libOS makes no such semantic-causation claim: its causal links encode durable, explicit structural relations among Runtime records. It also does not attempt whole-host provenance. The Explainable Operations view covers the recorded structural lineage from an LLM call through Tool, syscall, primitive, decision, Human, provider, resource, event, and audit records; a separate manifest records context materialization. Its completeness predicate explicitly identifies required roles and missing evidence and never treats an evidence graph as an authorization credential.

8.6 Agent security evaluation

Research on indirect prompt injection shows that untrusted observations can elicit harmful actions [14]. InjecAgent, ToolEmu, and AgentDojo provide benchmarks or environments for task-level security evaluation [6, 37, 58]. SWE-bench, WebArena, and OS-World measure useful behavior in software, web, and desktop environments [18, 55, 64]. The Agent libOS evaluation addresses a complementary question: whether authority, flow, recovery, and evidence invariants hold as the Runtime action surface changes. It reports deterministic boundary tests separately from real-model utility. This paper neither reports AgentDojo results nor claims that its synthetic live tasks replace broad task-success benchmarks.

9 Limitations and Conclusion

9.1 Limitations and trusted boundary

Not a kernel sandbox. Agent libOS runs inside a conventional Python process and relies on the Host OS and providers. Deno supervision, path checks, typed Git command mediation, and transport

bounds provide defense in depth. Trusted Runtime Modules and provider implementations, together with MCP stdio executables and authorized shell or native programs, can exercise Host authority outside the typed boundary. Deployments requiring stronger isolation should run such components in containers, WASM runtimes, or microVMs, or move them behind remote-service boundaries.

Prompt injection remains possible. An adversarial observation may alter the model’s plan, trigger repeated denied requests, consume budget, or induce a harmful action that is nevertheless permitted by the granted task policy. The three-plane design constrains mediated authority and delivery; it does not establish semantic correctness. Semantic Shadow is disabled by default, cannot provide a general allow predicate, and does not write classifier findings back into ambient labels.

Mediation has endpoints. Sink clearance applies only to deliveries mediated by the Runtime. Once a trusted shell, PTY, Human, LLM provider, MCP server, JSON-RPC service, filesystem target, or Git remote receives data, its subsequent handling, forwarding, and retention lie outside the threat model. Labels provide neither encryption nor deletion guarantees. The Runtime also cannot observe effects produced by Host code that bypasses its primitives.

Protocol and platform coverage is finite. The MCP implementation is a governed client subset rather than an implementation of the full specification. Typed Git is limited to a configured local repository and its existing remotes; it does not provide GitHub or GitLab product integration. PTY and Deno containment remain subject to platform-specific release gates. The local GUI is an operator console, not a hardened multi-tenant remote control plane. Each Task Run supervises a single process tree; Task Runs do not form a distributed workflow scheduler.

9.2 Conclusion

Self-evolving LLM agents need a stable boundary beneath prompts, Tool schemas, Skills, generated code, images, checkpoints, and remote-resource catalogs. The Agent libOS design separates this boundary into three parts: conjunctive operation admission, independent information-flow admission, and durable causal evidence that never becomes authority. A process’s model-visible action surface may evolve, but resource authority remains determined by Capabilities and any applicable explicit or derived Task Authority ceiling when a concrete primitive executes. Data may leave only through a Host-resolved Sink for which the clearance check passes; conditionally permitted high-sensitivity egress additionally requires an exact one-shot release.

The implementation applies this principle through persistent processes, Object Memory, Skill projection, syscall-mediated JIT Tools, typed providers, images, checkpoints, Task Runs, and ordered startup recovery, subject to the commit-bound limitations reported above. Its Protected Operation protocol makes provider ambiguity explicit: preparation is durably recorded before dispatch, settlement links the available evidence, and reopening never implies replay. The resulting system does not prove that an agent will reason safely. It provides a reference boundary that constrains and explains which requested effects an evolving agent may make real.

Availability

The implementation, documentation, benchmark harness and release are available at <https://github.com/yingqi-z20/Agent-libOS>.

Acknowledgments

Generative AI tools assisted with code development, manuscript organization, and language polishing; they are not listed as authors. The author reviewed and revised all AI-assisted material and remains responsible for the technical and implementation claims, references, and final manuscript.

References

- [1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020. URL <https://www.usenix.org/conference/nsdi20/presentation/agache>.
- [2] Adam Bates, Dave (Jing) Tian, Kevin R. B. Butler, and Thomas Moyer. Trustworthy Whole-System provenance for the Linux kernel. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 319–334, Washington, D.C., August 2015. USENIX Association. ISBN 978-1-939133-11-3. URL <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates>.
- [3] Sebastian Burckhardt, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, and Christopher S. Meiklejohn. Durable Functions: Semantics for stateful serverless. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA):133:1–133:27, October 2021. doi: 10.1145/3485510. URL <https://doi.org/10.1145/3485510>.
- [4] Edward Y. Chang and Longling Geng. SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. *Proceedings of the VLDB Endowment*, 18(12):4874–4886, 2025. doi: 10.14778/3750601.3750611. URL <https://doi.org/10.14778/3750601.3750611>.
- [5] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control, 2025. URL <https://arxiv.org/abs/2505.23643>. arXiv preprint.
- [6] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 82895–82920, 2024. doi: 10.52202/079017-2636. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- [7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning*, 2026. URL <https://arxiv.org/abs/2503.18813>. Presented at IEEE SaTML 2026; preprint first posted in 2025.
- [8] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, May 1976. doi: 10.1145/360051.360056. URL <https://doi.org/10.1145/360051.360056>.
- [9] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252. URL <https://doi.org/10.1145/365230.365252>.
- [10] Dawson R. Engler, M. Frans Kaashoek, and James O’Toole. Exokernel: An operating system architecture for application-level resource management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles*, pages 251–266. Association for Computing Machinery, 1995. doi: 10.1145/224057.224076. URL <https://doi.org/10.1145/224057.224076>.
- [11] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. AgentScope: A flexible yet robust multi-agent platform, 2024. URL <https://arxiv.org/abs/2402.14034>. arXiv preprint.
- [12] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=CTr3bovS5F>.

- [13] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, SIGMOD '87, pages 249–259, New York, NY, USA, 1987. Association for Computing Machinery. ISBN 0-89791-236-5. doi: 10.1145/38713.38742. URL <https://doi.org/10.1145/38713.38742>.
- [14] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISC '23, pages 79–90. Association for Computing Machinery, 2023. doi: 10.1145/3605764.3623985. URL <https://doi.org/10.1145/3605764.3623985>.
- [15] Norman Hardy. KeyKOS architecture. *ACM SIGOPS Operating Systems Review*, 19(4):8–25, 1985. doi: 10.1145/858336.858337. URL <https://doi.org/10.1145/858336.858337>.
- [16] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations. In *35th USENIX Security Symposium (USENIX Security 26)*, Baltimore, MD, August 2026. USENIX Association. URL <https://www.usenix.org/conference/usenixsecurity26/presentation/he-yu>.
- [17] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- [18] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- [19] Collin Lee, Seo Jin Park, Ankita Kejriwal, Satoshi Matsushita, and John Ousterhout. Implementing linearizability at large scale and low latency. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, pages 71–86, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 978-1-4503-3834-9. doi: 10.1145/2815400.2815416. URL <https://doi.org/10.1145/2815400.2815416>.
- [20] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems*, volume 36, pages 51991–52008, 2023. doi: 10.52202/075280-2264. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html.
- [21] Torsten Lodderstedt, John Bradley, Andrey Labunets, and Daniel Fett. Best current practice for OAuth 2.0 security. RFC 9700, RFC Editor, January 2025. URL <https://www.rfc-editor.org/info/rfc9700/>. BCP 240; Internet Best Current Practice.
- [22] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. Unikernels: Library operating systems for the cloud. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 461–472. Association for Computing Machinery, 2013. doi: 10.1145/2451116.2451167. URL <https://doi.org/10.1145/2451116.2451167>.
- [23] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM agent operating system. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=L4HHkCDz2x>.
- [24] Dirk Merkel. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014. URL <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
- [25] Mark Samuel Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, Baltimore, Maryland, USA, May 2006. URL <https://scholarship.library.jhu.edu/handle/1774.2/873>.
- [26] Model Context Protocol. Model Context Protocol Specification, July 2026. URL <https://modelcontextprotocol.io/specification/2026-07-28>. Protocol revision 2026-07-28.
- [27] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3C Recommendation REC-prov-dm-20130430, World Wide Web Consortium, April 2013. URL <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>. Editors.
- [28] Andrew C. Myers. JFlow: Practical mostly-static information flow control. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 228–241. Association for Computing Machinery, 1999. doi: 10.1145/292540.292561. URL <https://doi.org/10.1145/292540.292561>.
- [29] Andrew C. Myers and Barbara Liskov. A decentralized model for information flow control. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, pages 129–142. Association for Computing Machinery, 1997. doi: 10.1145/268998.266669. URL <https://doi.org/10.1145/268998.266669>.
- [30] Martin Odersky, Yaoyu Zhao, Yichen Xu, Oliver Bračevac, and Cao Nguyen Pham. Securing agents with tracked capabilities. In *Proceedings of the ACM Conference on AI and Agentic Systems*, pages 812–838. Association for Computing Machinery, 2026. doi: 10.1145/3786335.3813127. URL <https://doi.org/10.1145/3786335.3813127>.
- [31] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>. arXiv preprint.
- [32] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eysers, Margo Seltzer, and Jean Bacon. Practical Whole-System provenance capture. In *Proceedings of the 2017 Symposium on Cloud Computing*, SoCC '17, pages 405–418, New York, NY, USA, September 2017. Association for Computing Machinery. ISBN 978-1-4503-5028-0. doi: 10.1145/3127479.3129249. URL <https://doi.org/10.1145/3127479.3129249>.
- [33] Srinath Perera, Kaviru Hapuarachchi, Frank Leymann, and Rania Khalaf. Robust agent compensation (RAC): Teaching AI agents to compensate. In *Proceedings of the ACM Conference on AI and Agentic Systems*, pages 253–262. Association for Computing Machinery, 2026. doi: 10.1145/3786335.3813141. URL <https://doi.org/10.1145/3786335.3813141>.
- [34] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihao Zhao, Laurens Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng2O0Jjr>.
- [35] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, King Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution, 2025. URL <https://arxiv.org/abs/2505.20286>. arXiv preprint.
- [36] Justin Richer and Fabien Imbault. Grant negotiation and authorization protocol (GNAP). RFC 9635, RFC Editor, October 2024. URL <https://www.rfc-editor.org/info/rfc9635/>. IETF Proposed Standard.
- [37] Yangjun Ruan, Honghua Dong, Andrew Wang, Silvius Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=GcEWtMklua>.
- [38] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, January 2003. doi: 10.1109/JSAC.2002.806121. URL <https://doi.org/10.1109/JSAC.2002.806121>.
- [39] Andrei Sabelfeld and David Sands. Declassification: Dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009. doi: 10.3233/JCS-2009-0352. URL <https://doi.org/10.3233/JCS-2009-0352>.
- [40] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551, 2023. doi: 10.52202/075280-2997. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- [41] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. EROS: A fast capability system. In *Proceedings of the 17th ACM Symposium on Operating Systems Principles*, pages 170–185. Association for Computing Machinery, 1999. doi: 10.1145/319151.319163. URL <https://doi.org/10.1145/319151.319163>.
- [42] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. URL <https://arxiv.org/abs/2504.11703>. arXiv preprint, version 3 revised 14 May 2026.
- [43] Shoaib Ahmed Siddiqui, Radhika Gaonkar, Boris Köpf, David Krueger, Andrew Pavard, Ahmed Salem, Shruti Tople, Lukas Wutschitz, Menglin Xia, and Santiago Zanella-Béguelin. Permissive information-flow analysis for large language models. *Transactions on Machine Learning Research*, 2025. URL <https://openreview.net/forum?id=ufYRO8y3mr>.
- [44] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Alan Chan, and Alex Pentland. Position: AI agents need authenticated delegation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 82211–82231. PMLR, 2025. URL <https://proceedings.mlr.press/v267/south25a.html>.
- [45] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira da Silva. PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows. In *2025 IEEE International Conference on eScience*, pages 467–473. IEEE, September 2025. ISBN 979-8-3315-9145-8. doi: 10.1109/eScience65000.2025.00093. URL <https://ieeexplore.ieee.org/document/11181558/>.
- [46] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A security architecture for governing AI agentic systems. In *Network and Distributed System Security Symposium 2026*, San Diego, CA, USA,

- February 2026. Internet Society. ISBN 979-8-9919276-8-0. doi: 10.14722/ndss.2026.230869. URL <https://www.ndss-symposium.org/ndss-paper/saga-a-security-architecture-for-governing-ai-agentic-systems/>.
- [47] Lillian Tsai and Eugene Bagdasarian. Contextual agent security: A policy for every purpose. In *Proceedings of the Workshop on Hot Topics in Operating Systems, HOTOS '25*, pages 8–17, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 979-8-4007-1475-7. doi: 10.1145/3713082.3730378. URL <https://doi.org/10.1145/3713082.3730378>.
- [48] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=ehfRiF0R3a>.
- [49] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, pages 1–12. Association for Computing Machinery, 2026. ISBN 979-8-4007-2025-3. doi: 10.1145/3744916.3764546. URL <https://conf.researchr.org/details/icse-2026/icse-2026-research-track/29/AgentSpec-Customizable-Runtime-Enforcement-for-Safe-and-Reliable-LLM-Agents>.
- [50] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 50208–50232. PMLR, 2024. URL <https://proceedings.mlr.press/v235/wang24h.html>.
- [51] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=OJd3ayDDoF>.
- [52] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: Practical capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*. USENIX Association, 2010. URL <https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix>.
- [53] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024. URL <https://arxiv.org/abs/2409.19091>. arXiv preprint.
- [54] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
- [55] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37, pages 52040–52094, 2024. doi: 10.52202/079017-1650. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html.
- [56] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652, 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.
- [57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [58] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.624. URL <https://aclanthology.org/2024.findings-acl.624/>.
- [59] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1187–1204. USENIX Association, November 2020. ISBN 978-1-939133-19-9. URL <https://www.usenix.org/conference/osdi20/presentation/zhang-haoran>.
- [60] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=pUpzQZTvGY>.
- [61] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=z5uVAKwmjf>.
- [62] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. SkillWeaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>. arXiv preprint.
- [63] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. AgentSight: System-level observability for AI agents using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems, PACMI '25*, pages 110–115, New York, NY, USA, October 2025. Association for Computing Machinery. ISBN 979-8-4007-2205-9. doi: 10.1145/3766882.3767169. URL <https://doi.org/10.1145/3766882.3767169>.
- [64] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oKn9c6ytLx>.
- [65] Siyuan Zhuang, Stephanie Wang, Eric Liang, Yi Cheng, and Ion Stoica. ExoFlow: A universal workflow system for Exactly-Once DAGs. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 269–286, Boston, MA, July 2023. USENIX Association. ISBN 978-1-939133-34-2. URL <https://www.usenix.org/conference/osdi23/presentation/zhuang>.